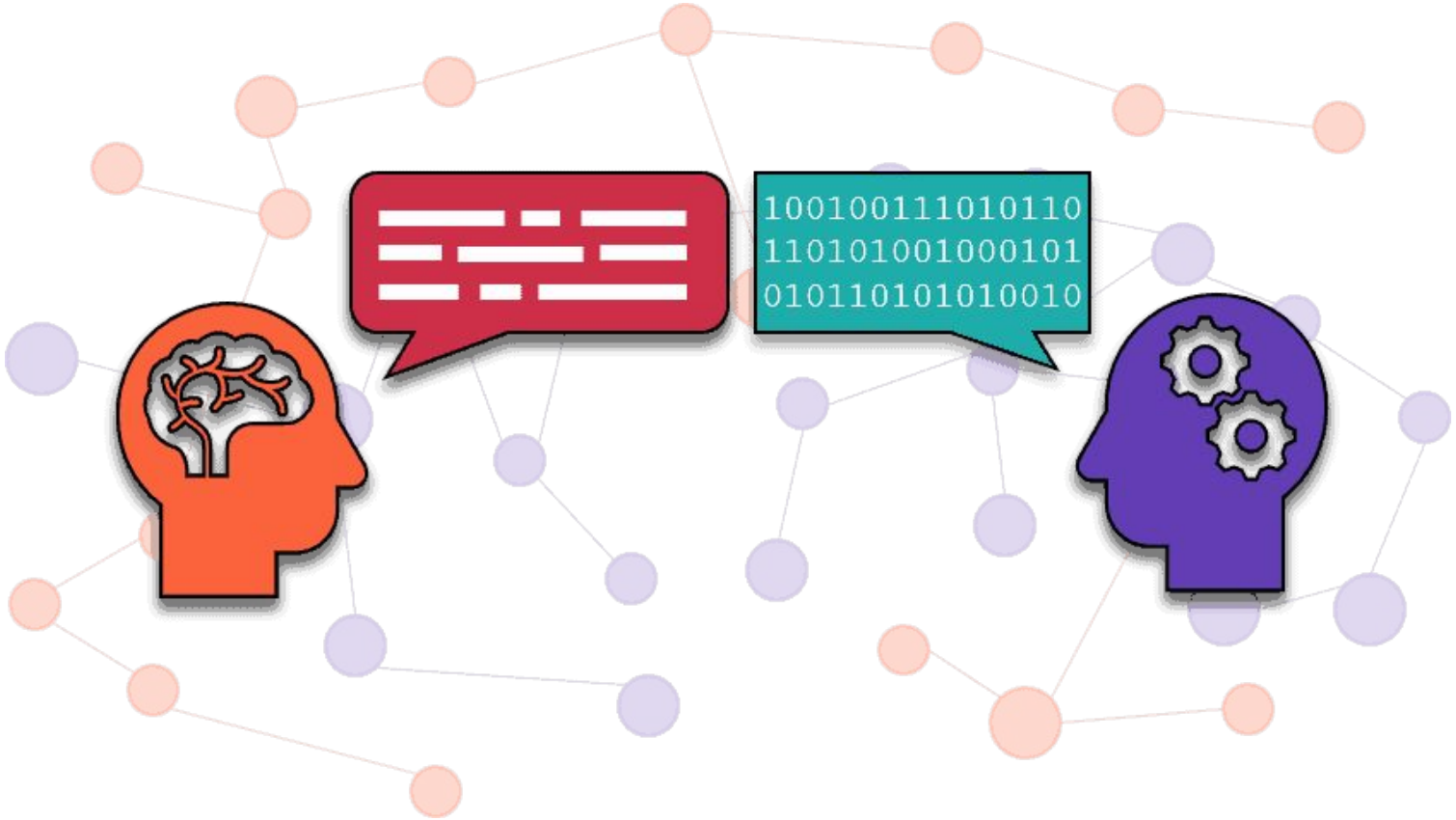
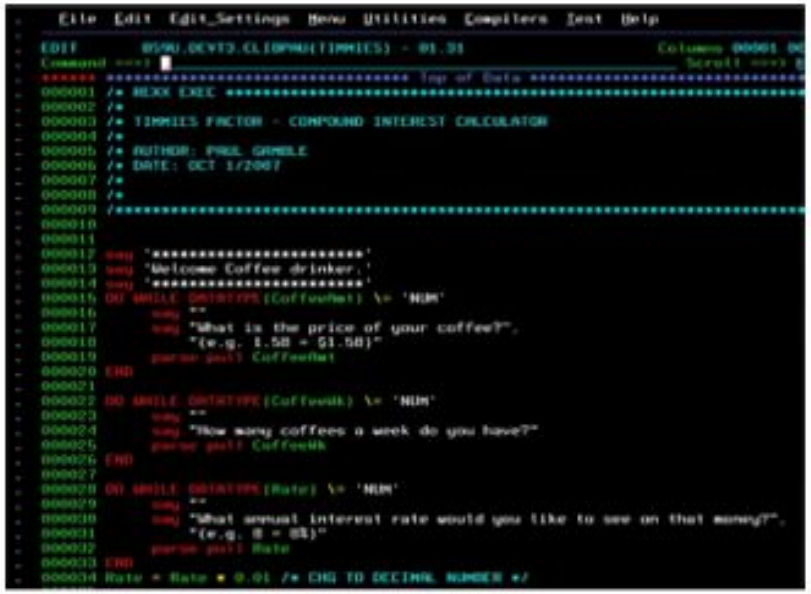
Artificial Intelligence and Machine Learning
6CS012



Week 08 Lecture : Introduction to NLP and Text Representation Techniques

Communication with Machines

~50-70s

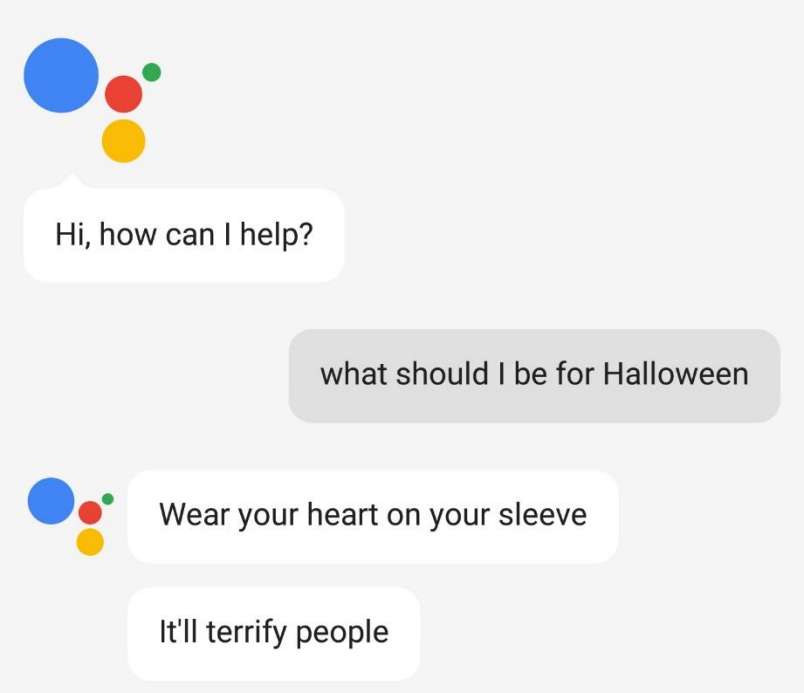


~80s

today

Conversational agents

- Speech recognition
- Language analysis
- Dialogue processing



Applications

Information Retrieval

Machine Translation

Natural Language Processing

Applications:

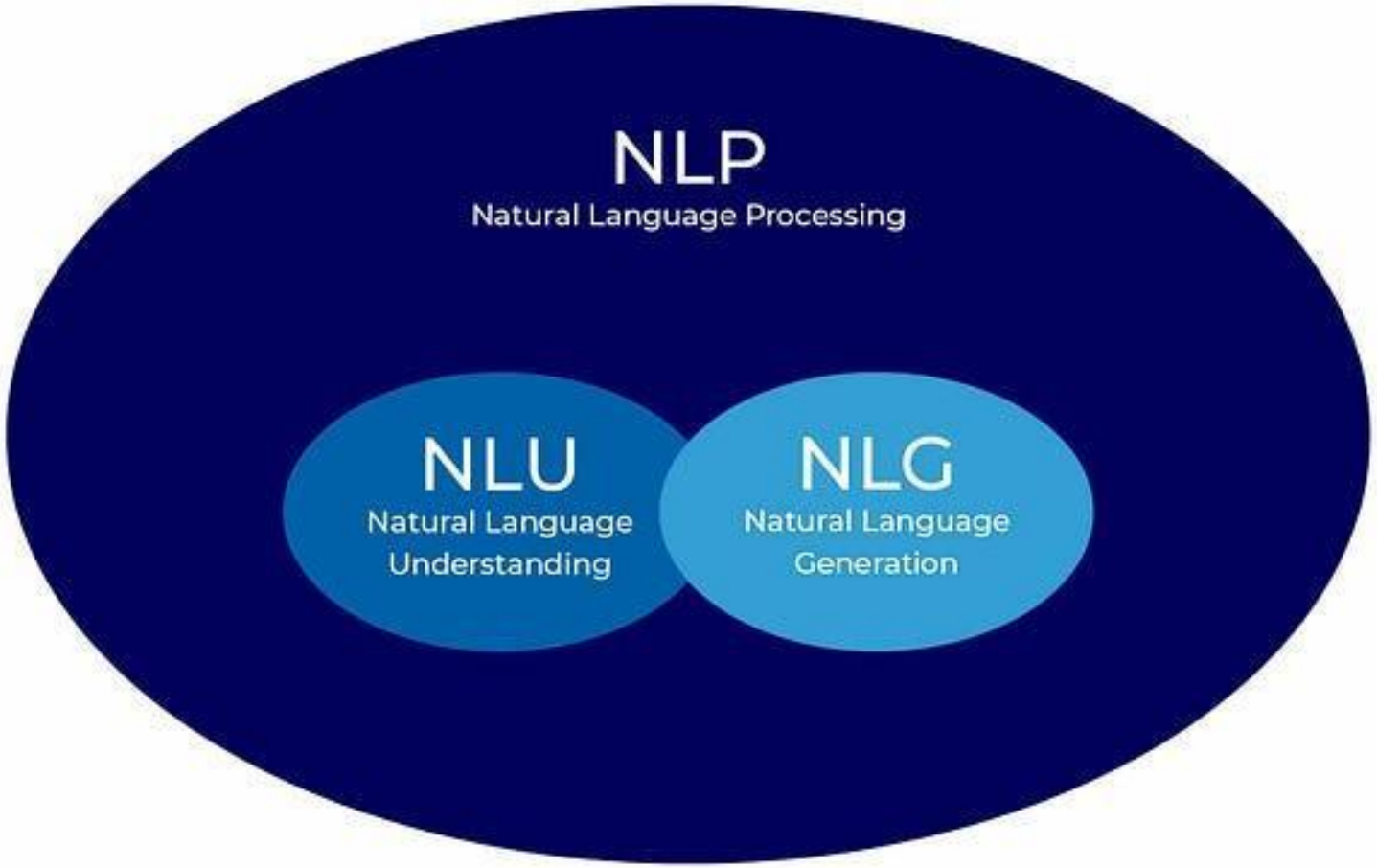
- Machine Translation
- Information Retrieval
- Question Answering
- Dialogue Systems
- Information Extraction
- Summarization
- Sentiment Analysis etc.

Core Technologies

- Language modeling
- Part-of-speech tagging
- Syntactic parsing
- Named-entity recognition
- Word sense disambiguation
- Semantic role labeling

NLP lies at the intersection of computational linguistics and machine learning!!

Natural Language Processing



Level of Linguistic Knowledge

- Phonetics, Phonology:

SOUNDS Th i a si e n

- Words

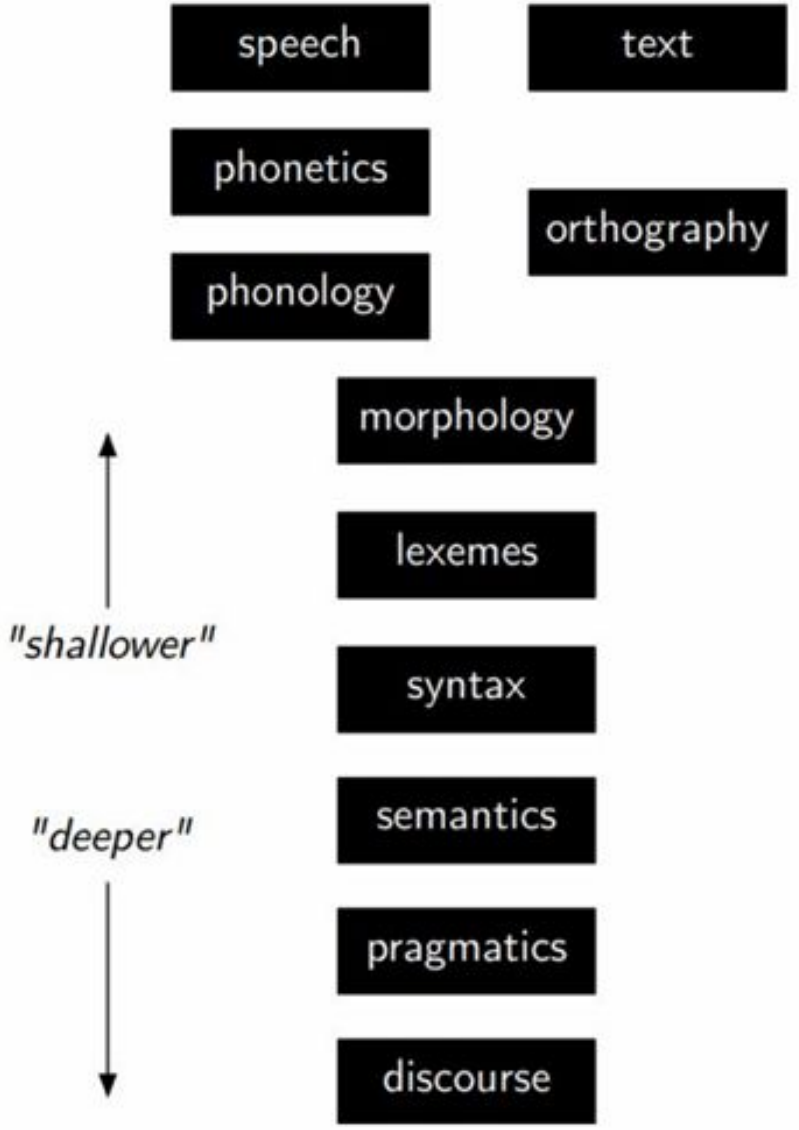
This is a simple sentence

- Morphology

Word Formations (root, affixes etc.)

This is a simple sentence

be

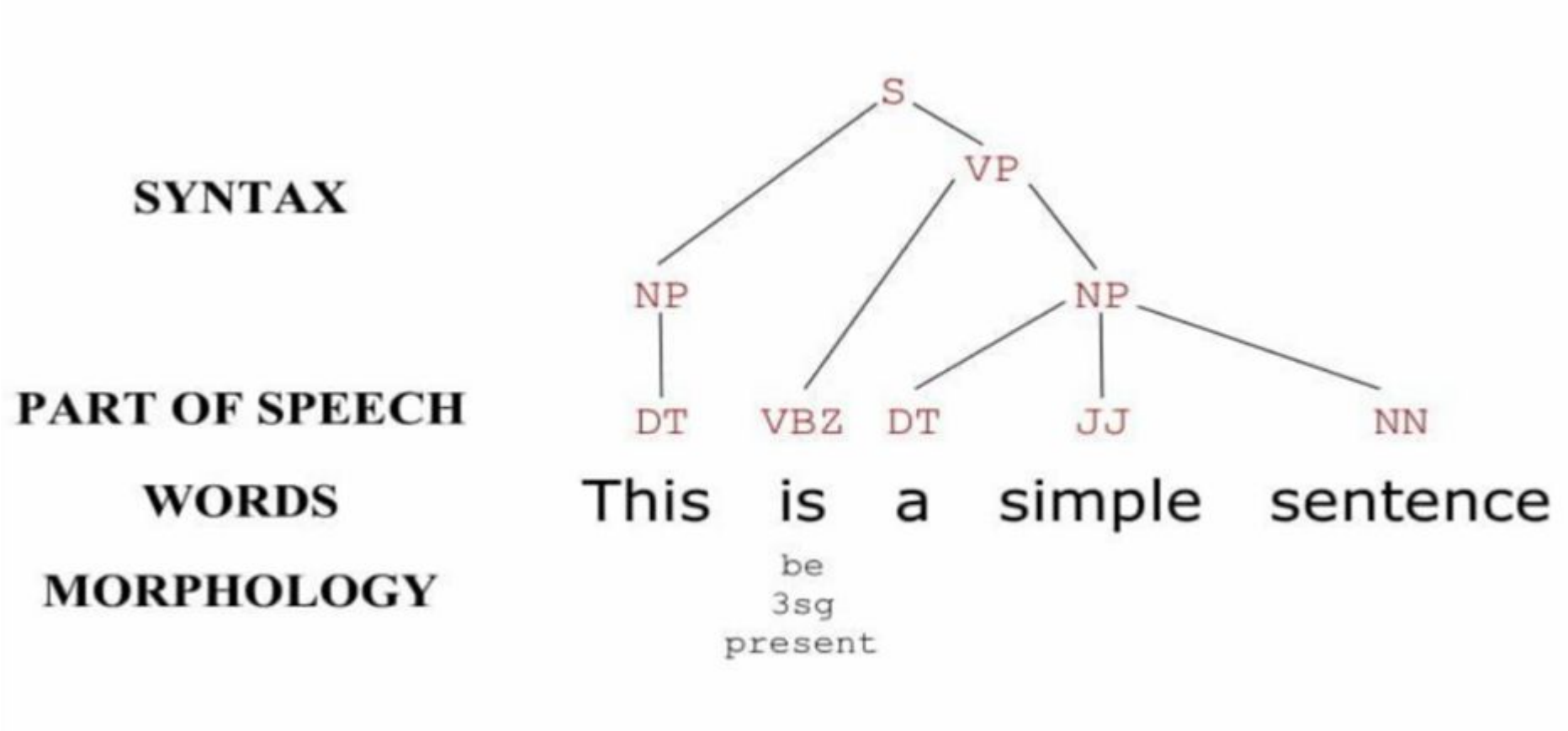


Part of Speech (PoS)

- Part of Speech tagging

Syntax

- Syntactic Parsing



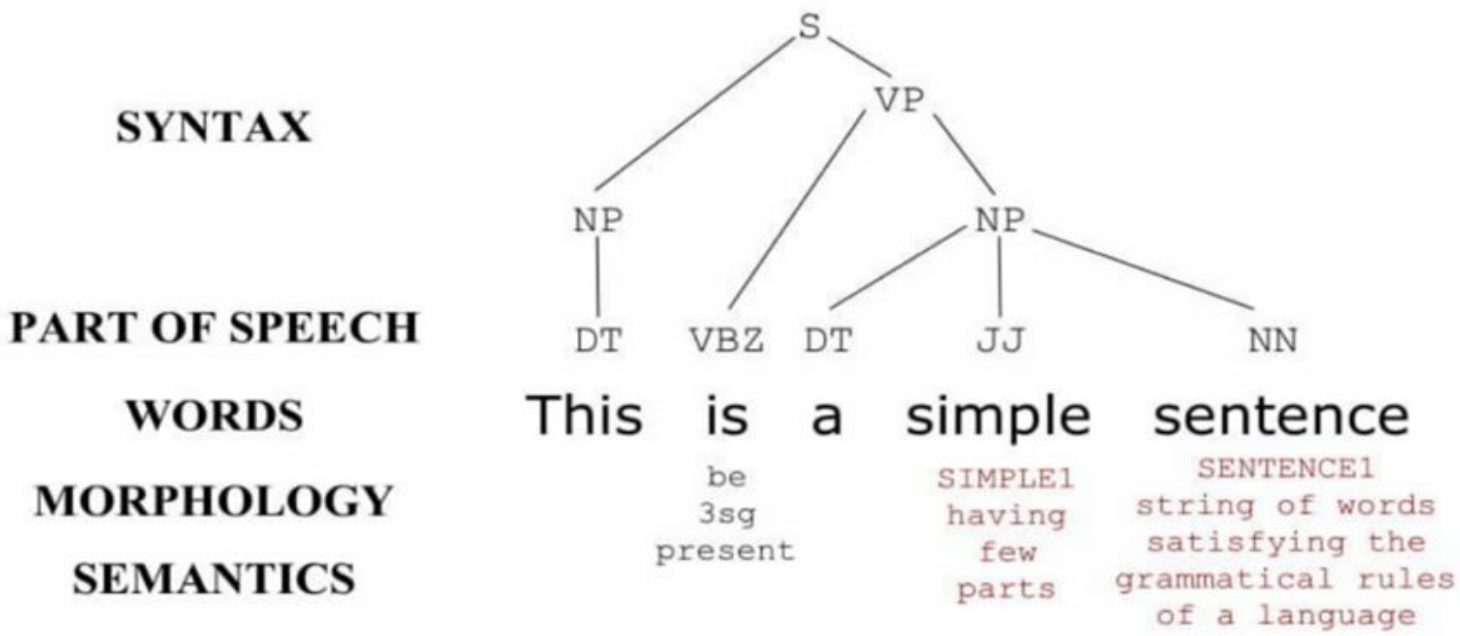
Semantics /Pragmatics

Meaning in language and How context contributes to meaning beyond literal interpretation

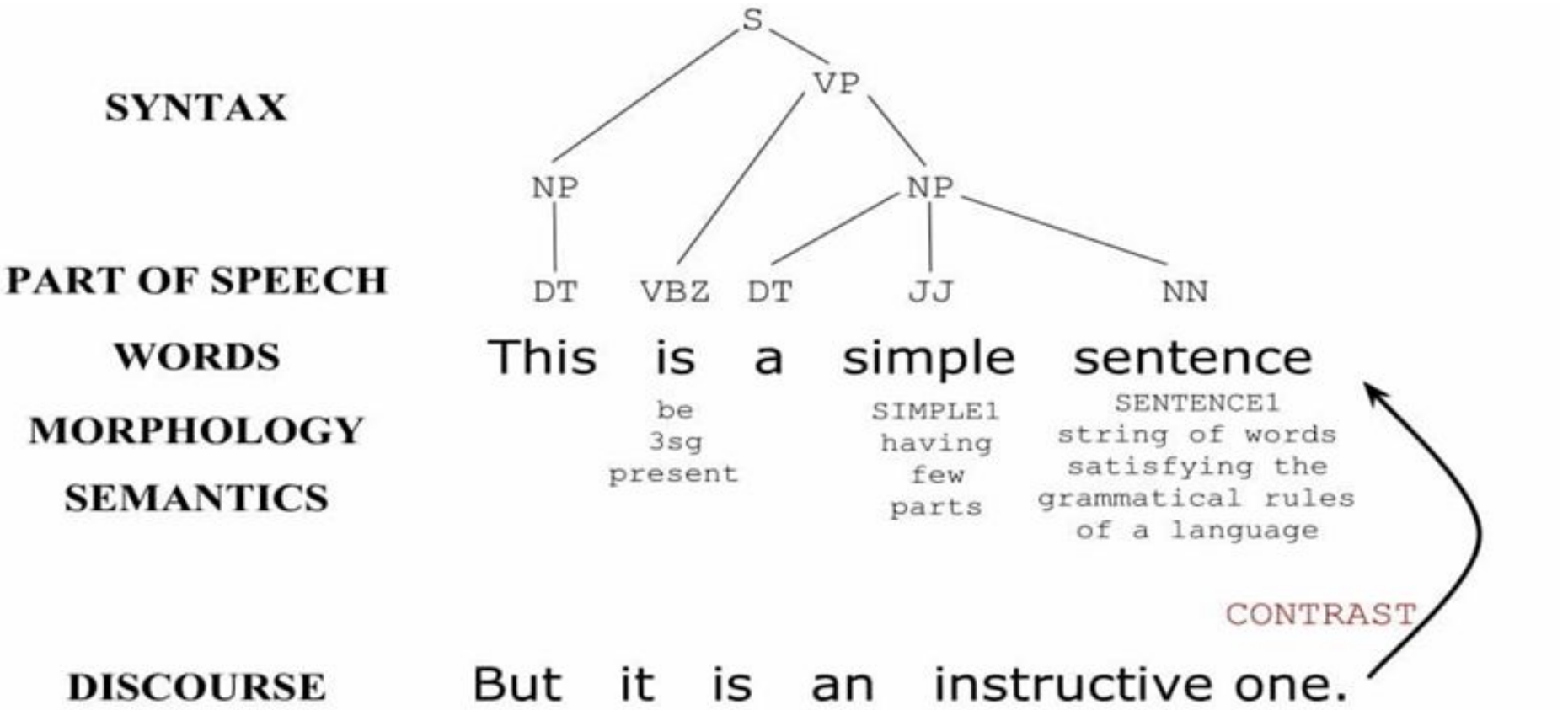
- Named entity recognition
- Word sense disambiguation
- Semantic role labelling

The square root of Thursday is Purple!

Her heart was like stone!



Discourse

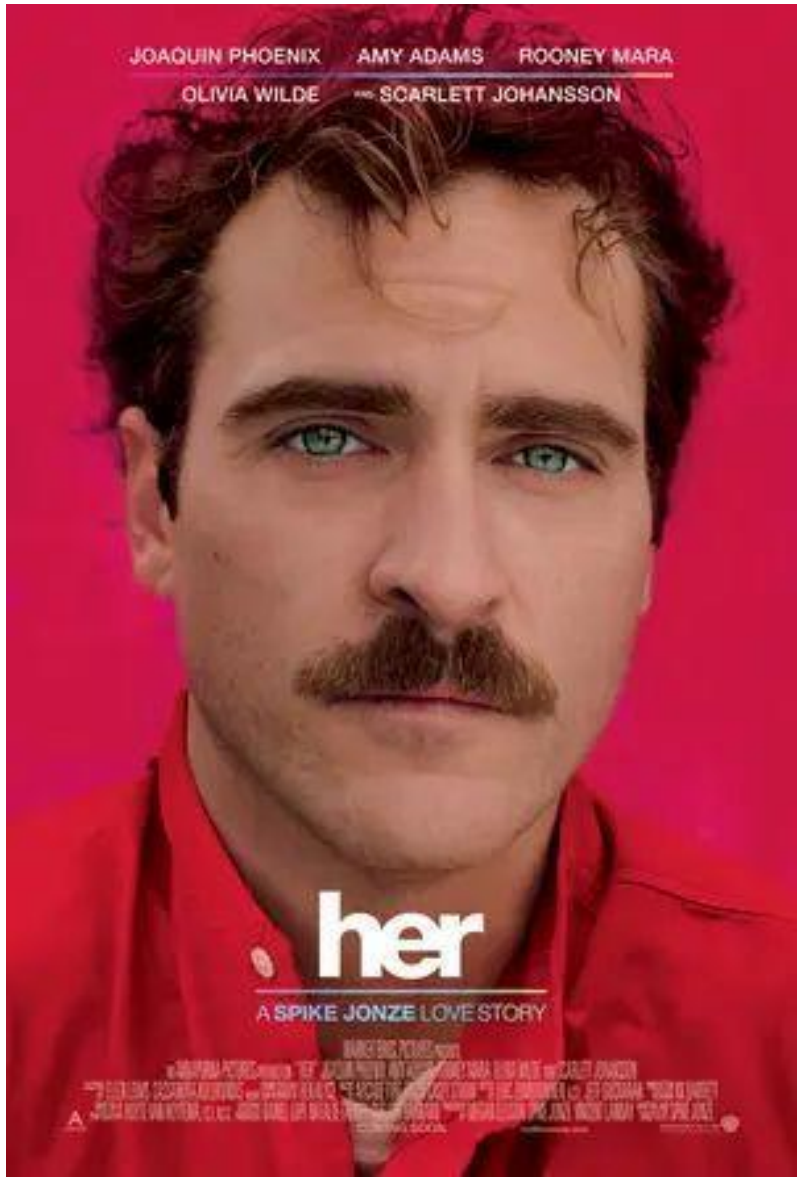

SYNTAX
S
VP
NP
NP
PART OF SPEECH
DT
VBZ
DT
JJ
NN
WORDS
This is a simple sentence
MORPHOLOGY
be
3sg
present
SIMPLE1
having
few
parts
SENTENCE1
string of words
satisfying the
grammatical rules
of a language
SEMANTICS
CONTRAST
DISCOURSE
But it is an instructive one.

Where are we Now?

Claude

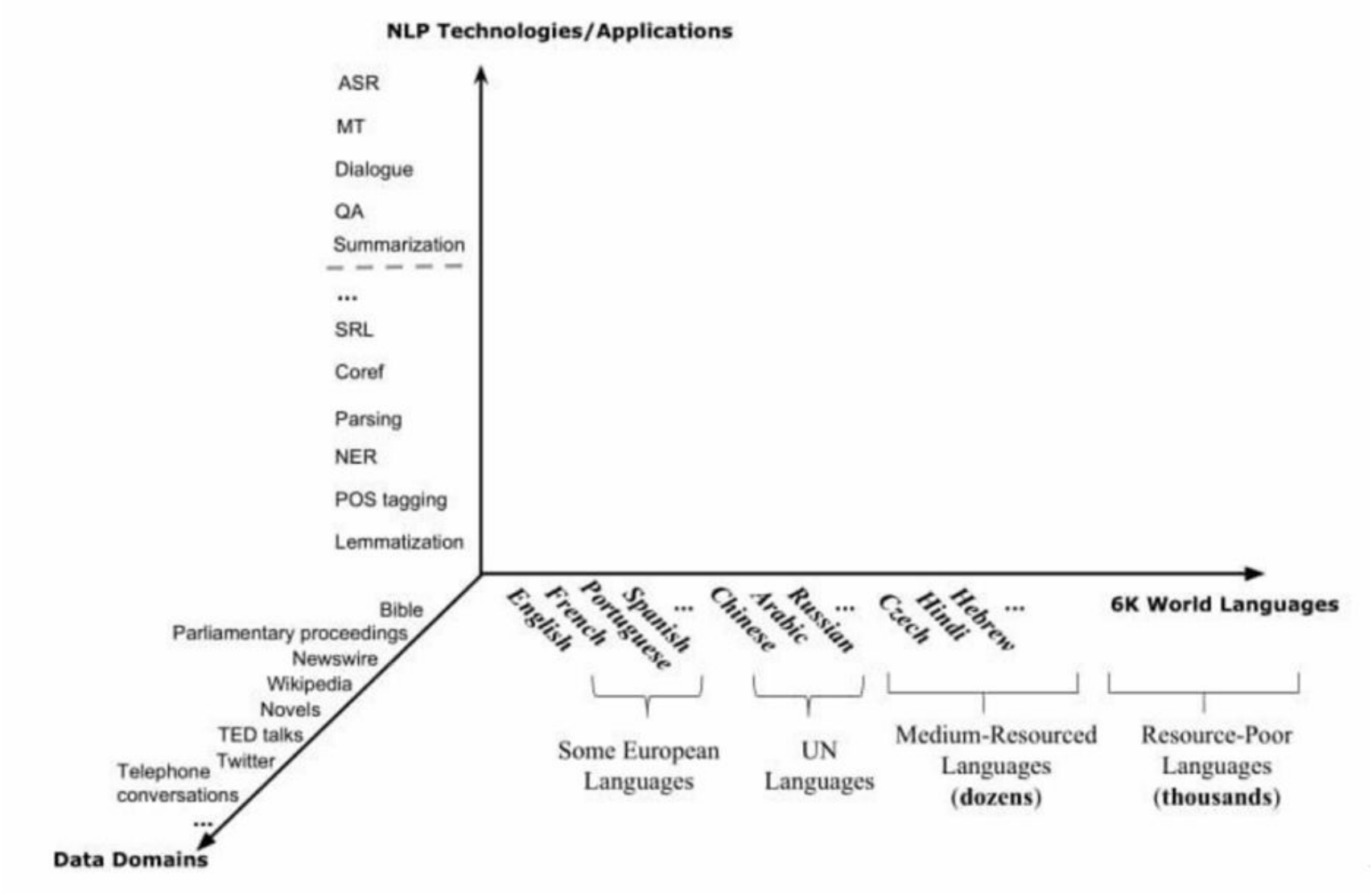
Gemini

VS



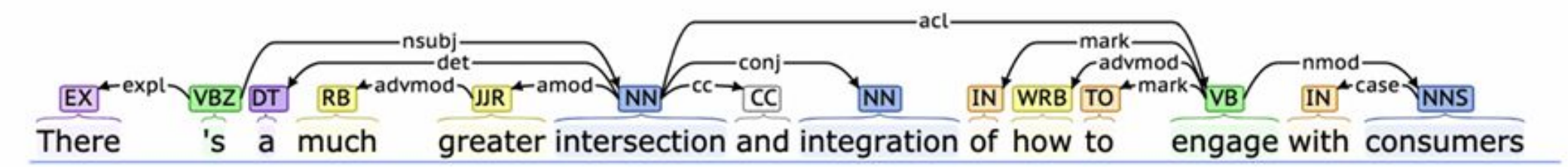
Why is NLP hard?

Language Variation



Language is not static (variation)

Suppose we train a part of speech tagger or a parser on the Wall Street Journal



What will happen if we try to use this tagger/parser for social media?

I know, right — shake my head — for — your

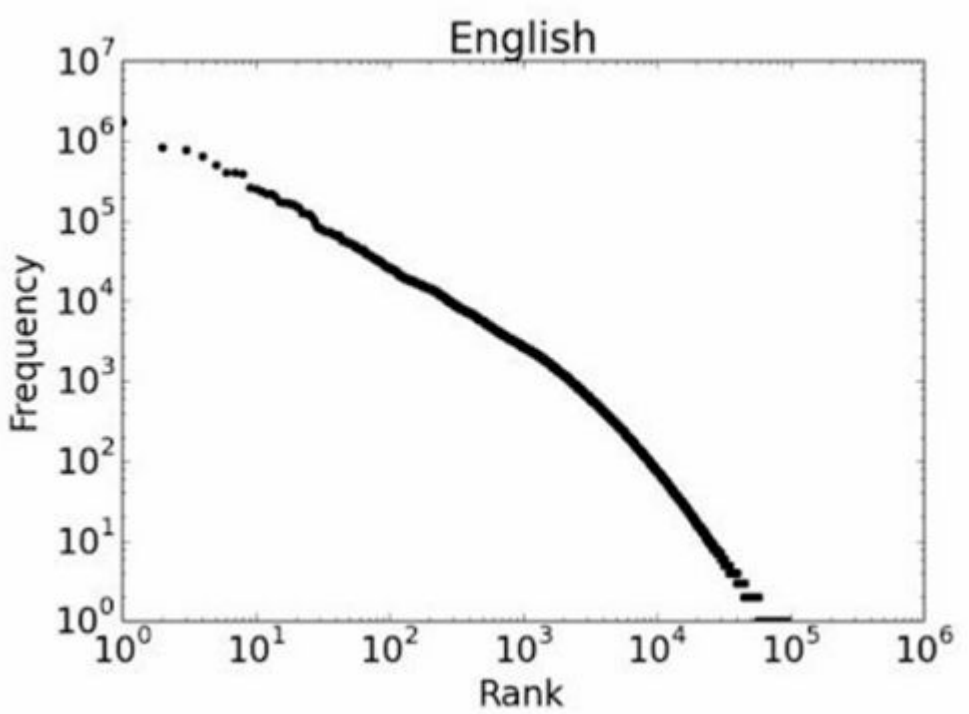
ikr smh he asked fir yo last name

you — Facebook — laugh out loud

so he can add u on fb lolololol

Scale and Sparsity

- Bible (King James version): ~700K
- Newswire collection: 500M+
- Wikipedia: 2.9 billion word (English)
- Web: several billions of words



Regardless of how large our corpus is, there will be a lot of infrequent words

This means we need to find clever ways to estimate probabilities for things we have rarely or never seen

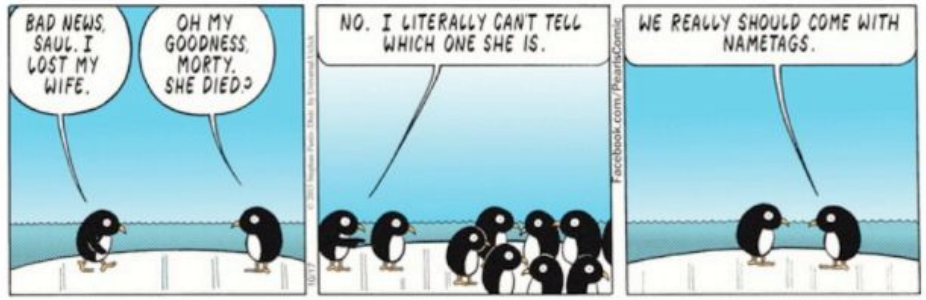
Ambiguity at multiple Levels

Ambiguity at multiple levels :

Word senses: bank (finance or river ?)

Syntactic structure: I can see a man with a telescope.

Multiple meaning: I made her duck



credit: A. Zwicky

Ambiguity at multiple Levels

- Part of speech Tagging
- Pronoun reference

Dr. Macklin often brings his dog Champion to visit with the patients. He just loves to give big, wet, sloppy kisses!

San Jose cops kill man with knife

Text Paper Translate Listen Close

San Jose cops kill man with knife

Ex-college football player, 23, shot 9 times allegedly charged police at fiancee's home

By Hamed Aleaziz and Vivian Ho

A man fatally shot by San Jose police officers while allegedly charging at them with a knife was a 23-year-old former football player at De Anza College in Cupertino who was distraught and depressed, his family said Thursday.

Police officials said two officers opened fire Wednesday afternoon on Phillip Watkins outside his fiancee's home because they feared for their lives. The officers had been drawn to the home, officials said, by a 911 call reporting an armed home invasion that, it turned out, had been made by Watkins himself.

But the mother of Watkins' fiancee, who also lives in the home on the 4300 block of Sherman Street, said she witnessed the shooting and described it as excessive. Faye Buchanan said the confrontation happened shortly after she called a suicide intervention hotline in hopes of getting Watkins medical help.

Watkins' 911 call came in at 5:01 p.m., said Sgt. Heather Randol, a San Jose police spokeswoman. "The caller stated there was a male breaking into his home armed with a knife," Randol said. "The caller also stated he was locked in an upstairs bedroom with his children and requested help from police."

She said Watkins was on the sidewalk in front of the home when two officers got there. He was holding a knife with a 4-inch blade and ran toward the officers in a threatening manner, Randol said.

"Both officers ordered the suspect to stop and drop the knife," Randol said. "The suspect continued to charge the officers with the knife in his hand. Both officers, fearing for their safety and defense of their life, fired at the suspect."

On the police radio, one officer said, "We have a male with a knife. He's walking toward us."

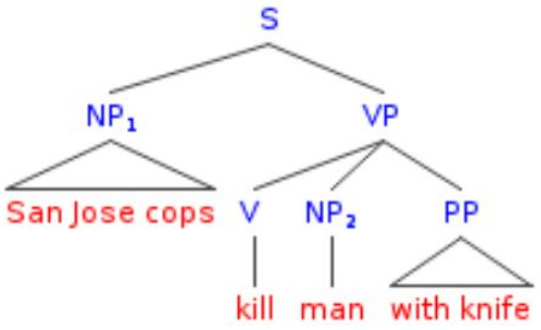
"Shots fired! Shots fired!" an officer said moments later.

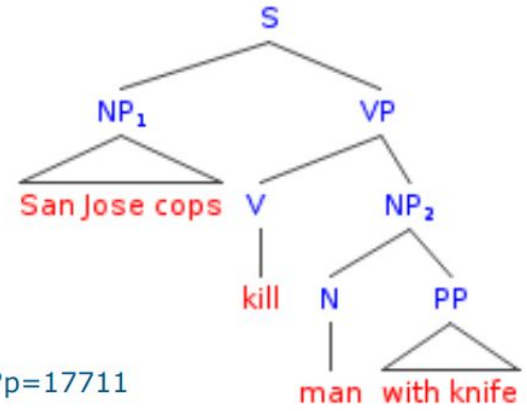
A short time later, an officer reported, "Male is down. Knife's still in hand."

Buchanan said she had been prompted to call the

Shoot continues on D8

Back Continue





Credit: Mark Liberman, http://languagelog.ldc.upenn.edu/nll/?p=17711

Text Representation

How to represent texts in numerical form?

Representing Text as Symbol

- Map each word to unique Id
- Example mappings:

  'A' → 65

  'a' → 97'0' → 48

  ' ' (space) → 32

Character	ASCII Code	Binary
A	65	01000001
B	66	01000010
C	67	01000011

Not a good idea!!!

Why?

- The representation is character – wise

- The representation cannot incorporate semantic information of words

  Computing Similarity – these fixed representations must also be semantically similar

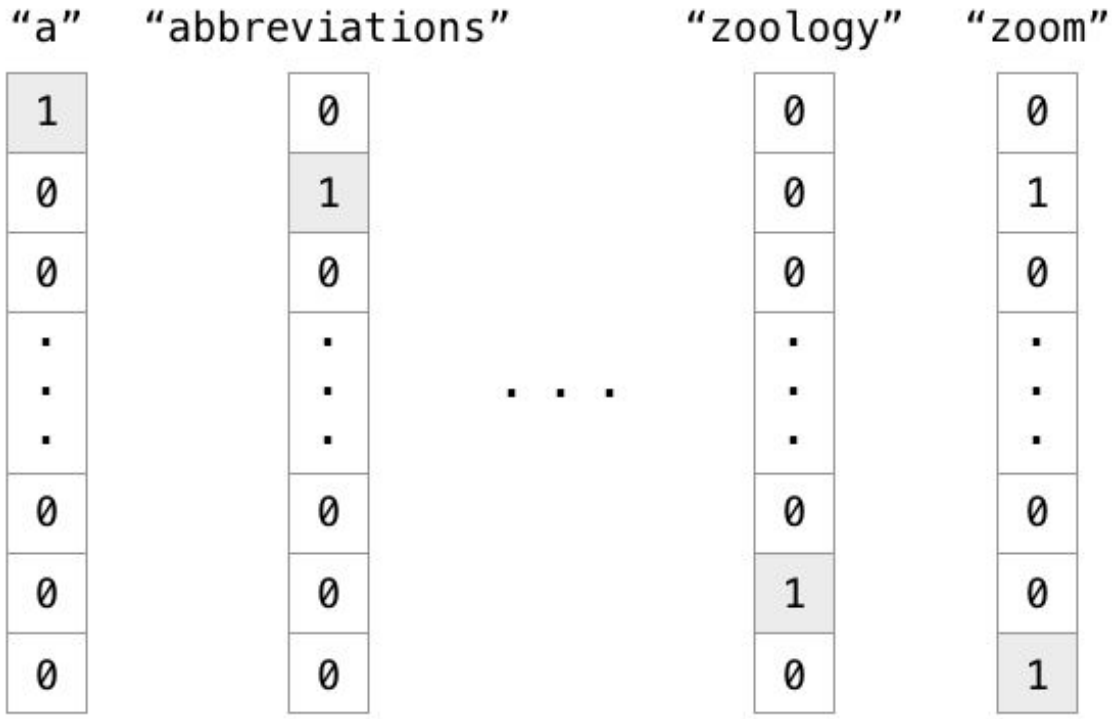
"desk" : [01100100, 01100101, 01110011, 01101011]
length = $4 \times 8 = 32$ → {Modern ASCII uses 8-bit character encoding}

"table" : [01100100, 01100101, 01110011, 01101011]
length = $5 \times 8 = 40$ → {Modern ASCII uses 8-bit character encoding}
adding one character increases the length by 8.

One hot Encoding

- Represent each word as a **unique ID**, then encode it as a **one-hot vector**.
- A vector filled with 0s, except for a 1 at the position corresponding to the word's ID.
- Example: Vocabulary size D=10, word w with ID=4:

  e(w) = [0 0 0 1 0 0 0 0 0 0]



Solved the variable length problem but makes no assumptions about similarity!!

Drawbacks

- **Scalability & Efficiency**

  Input size = vocabulary size → impractical for large/unknown vocabularies (Zipf's law).

  Computationally expensive: sparse vectors with *O(D)* parameters.

  OOV Problem: Requires "UNK" token for rare/unseen words.

- **No Semantic Relationships**

  All words are orthogonal:

  D(cat, refrigerator) = *D*(cat, cats)

  D(spoon, knife) = *D*(spoon, dog)

Bag of Words

Represents text as an **unordered set of words**, ignoring grammar/order but keeping frequency.

Documents:

"The cat sat on the mat."

"The dog ate the cat’s food."

Vocabulary: [the, cat, sat, on, mat, dog, ate, food, ’s]

BoW Vectors:

Doc1: [2, 1, 1, 1, 1, 0, 0, 0, 0]

Doc2: [2, 1, 0, 0, 0, 1, 1, 1, 1]

- **Drawbacks:**
- **No Semantics**: "cat" and "dog" treated as unrelated.
- **No Order**: "cat eats dog" vs. "dog eats cat" → identical BoW.
- **High Dimensionality**: Grows with vocabulary size.
- **Sparsity**: Mostly zeros for large vocabularies.

Term Frequency (TF)

- How important is a term(t) within one document(d)?

$$\mathrm{TF}(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total terms in } d}$$

- Document d_1:

  "AI is the future and the future is now“

  TF of "future" = 2 / 8 = 0.25

  TF of "the" = 2 / 8 = 0.25

  TF of "AI" = 1 / 8 = 0.125

Limitations

Diminishing Returns:

Seeing a term 10× doesn't make it 10× more meaningful.

No Normalization:

Raw TF favors longer documents with repeated words.

Log Scaled TF weighting

$$\mathrm{tf}_{\mathrm{weighted}}(t,d) = \begin{cases} 1 + \log_{10}(f_{t,d}) & \text{if } f_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$$

"AI is the future and the future is now"

Term	Raw TF $f_{t,d}$	Weighted TF $1 + \log_{10}(f_{t,d})$
future	2	$1 + \log_{10}(2) \approx 1.301$
the	2	$1 + \log_{10}(2) \approx 1.301$
AI	1	1.0
is	2	$1 + \log_{10}(2) \approx 1.301$
and	1	1.0
now	1	1.0

Reduces impact of high-frequency term!!!

Inverse Document Frequency (IDF)

How rare or common is a term across all documents? (the term is discriminative only if it is rare)!

$$IDF(t) = \log\left(\frac{\text{Total documents}}{\text{Documents containing } t}\right)$$

Corpus:
1."The cat sat..."
2."The dog ate..."
3."The bird sang..."

Term	Docs Containing Term	IDF (log(3/x))
the	3	log(3/3)=0
cat	1	log(3/1)≈1.098
bird	1	log(3/1)≈1.098

Why It Matters:
- "The" has IDF=0 → filtered out as noise.
- "Cat" and "bird" have high IDF → key terms for search/classification.

TF-IDF

- Highlights important, non-common words
- Reduces weight of frequent but uninformative words ("the", "is", etc.)

$$TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t)$$

"the cat sat on the mat"

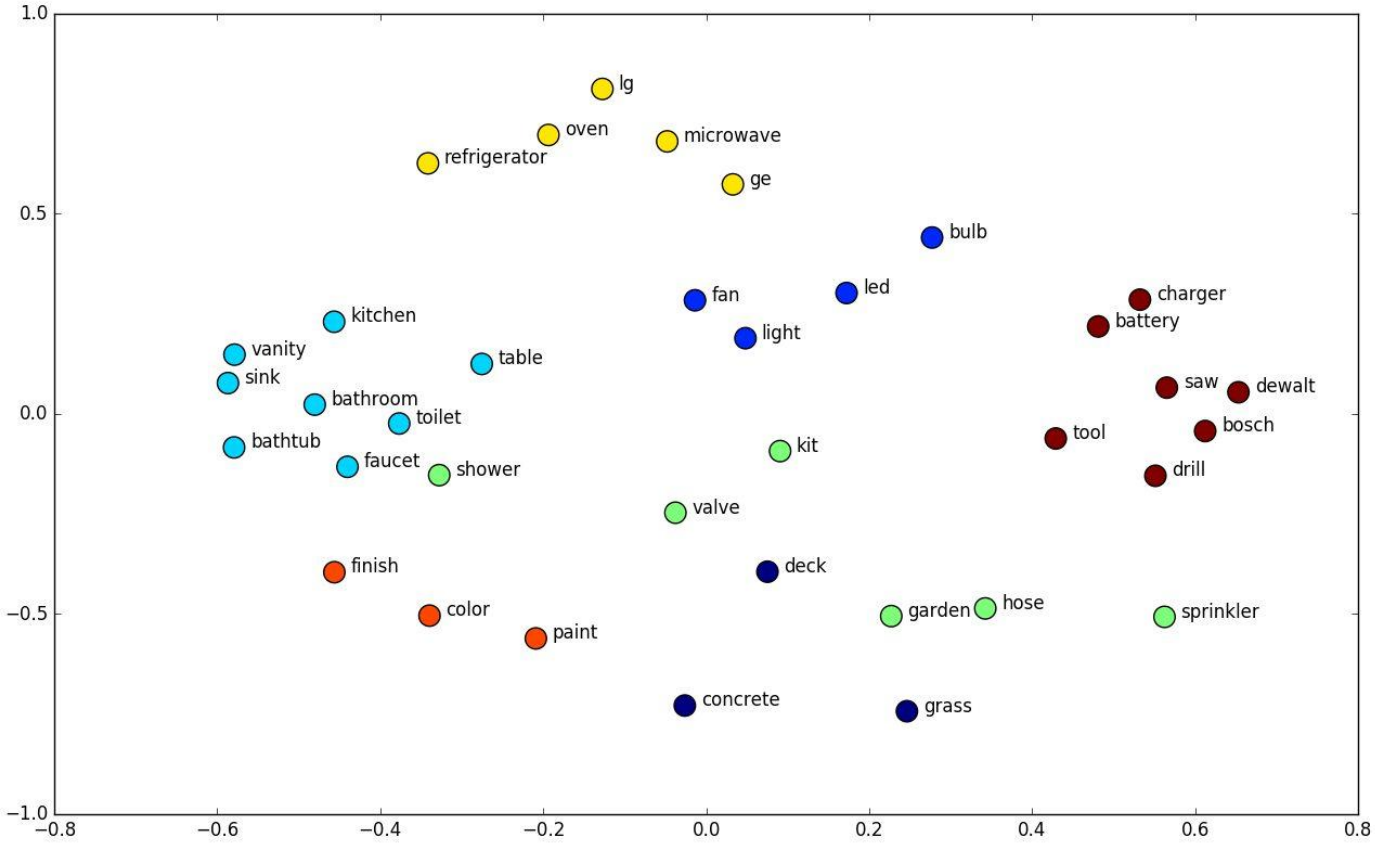
"the dog ate the cat's food"

"the bird sang"

Term	Raw TF (Doc1)	TF Weighted (Doc1)	IDF (log(3/df))	TF-IDF (Doc1)
the	2	1 + log(2) ≈ 1.301	log(3/3) = 0	1.301 × 0 = 0
cat	1	1 + log(1) = 1.0	log(3/2) ≈ 0.176	1.0 × 0.176 ≈ 0.176
sat	1	1.0	log(3/1) ≈ 0.477	1.0 × 0.477 ≈ 0.477
dog	0	0.0	log(3/1) ≈ 0.477	0.0 × 0.477 = 0
bird	0	0.0	log(3/1) ≈ 0.477	0.0 × 0.477 = 0

Limitations

- **No Semantic Understanding**
  Treats words as unique tokens, ignoring word meaning (e.g., “car” ≠ “automobile”).
- **Sparsity & High Dimensionality**
  Creates large, sparse vectors equal in length to vocabulary size.
- **No Generalization / Fixed Vocabulary**
  Cannot handle out-of-vocabulary (OOV) words; adding new terms requires recomputation.
- **Ignores Word Order & Context**
  Works as a bag-of-words model, losing syntax and negation (e.g., “good” vs. “not good”).
- **Hard to Compare Across Corpora**
  IDF values depend on the specific corpus, reducing comparability.



Representing the language semantics (word Vectors)

Distributional Representation

"You shall know a word by the company it keeps" (Firth, 1957)

- Compact and dense low dimensional representation of words
- Distributional Hypothesis:

  "Linguistic Units with similar distributions have similar meaning"

- Meaning is defined by the context it appears
- Semantic properties:

  Individual vector components are **not meaningful** in isolation

  Meaning is distributed across all dimensions

Word Embedding: Inspiration.

- **Experiment 1: based on “tezguino” example by Lin in 1998.**

Do you know what the word tezgüino means ?

(We hope you do not)

Word Embedding: Inspiration.

Now look how this word is used in different contexts:

A bottle of tezgüino is on the table.

Everyone likes tezgüino.

Tezgüino makes you drunk.

We make tezgüino out of corn.

Can you understand what tezgüino means ?

How did you do this?

Word Embedding: Inspiration.

Now look how this word is used in different contexts:

A bottle of tezgüino is on the table.

Everyone likes tezgüino.

Tezgüino makes you drunk.

We make tezgüino out of corn.

Can you understand what tezgüino means ?

Context makes it easier to understand the words!!

Word Embeddings: Inspiration.

- **Experiment 2: What other words could fit into these context?**

(1) A bottle of _____ is on the table.

(2) Everyone likes _____ .

(3) _____ makes you drunk.

(4) We make _____ out of corn.

Word Embedding: Inspiration.

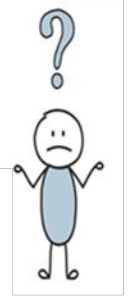
- **Experiment 2: What other words could fit into these context?**

(1) A bottle of _____ is on the table.

(2) Everyone likes _____ .

(3) _____ makes you drunk.

(4) We make _____ out of corn.



	(1)	(2)	(3)	(4)
tezgüino	1	1	1	1
loud	0	0	0	0
motor oil	1	0	0	1
tortillas	0	1	0	1
wine	1	1	1	0

Contexts

1 if a word appear in the context;
0 otherwise

Word Embedding: Inspiration.

- **Experiment 2: What other words could fit into these context?**

(1) A bottle of _____ is on the table.

(2) Everyone likes _____ .

(3) _____ makes you drunk.

(4) We make _____ out of corn.

	(1)	(2)	(3)	(4)
tezgüino	1	1	1	1
loud	0	0	0	0
motor oil	1	0	0	1
tortillas	0	1	0	1
wine	1	1	1	0

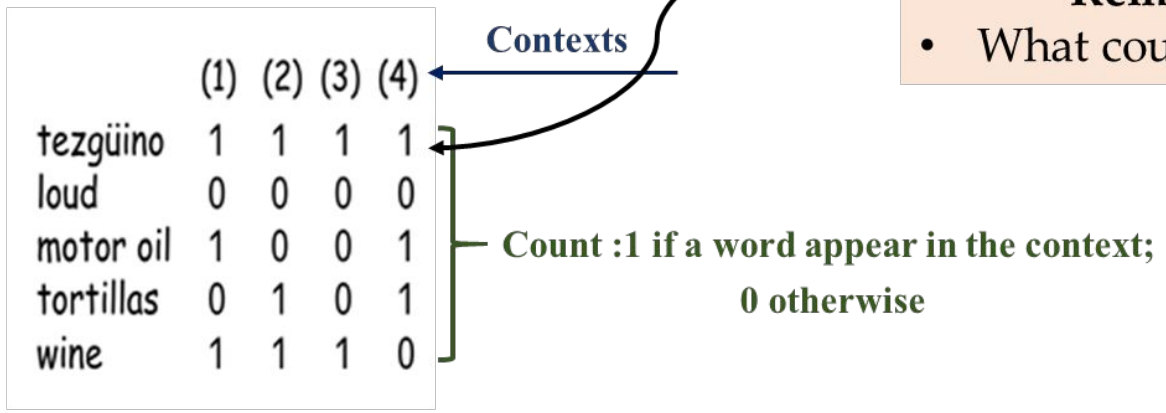
Contexts

Count :1 if a word appear in the context;
0 otherwise

Observations: Words with similar row {they appear in similar context} they may have similar meanings.

Word Embedding: Summary.

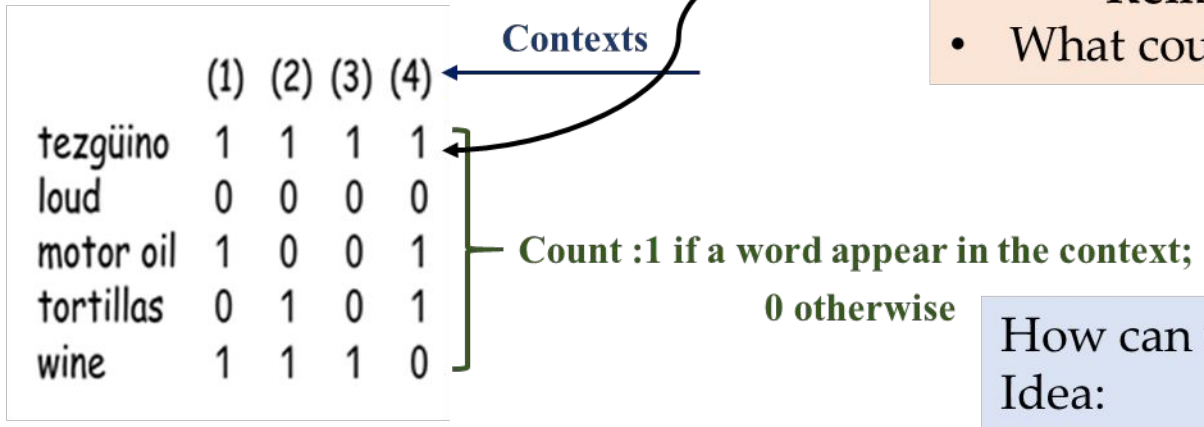
- Conclusion from "tezguino" experiment:
  - Observation 1: **context makes it easier to understand a word's meaning.**
  - Observation 2: **words with similar row {they appear in similar context} they may have similar meanings.**
- Problem to solve:



- How can we fill this matrix?
  - Is Count a good idea?
  - **Remember the limitations of TF – IDF, so no.**
- What could be the solutions?

Word Embeddings: Summary.

- Conclusion from "tezguino" experiment:
  - Observation 1: **context makes it easier to understand a word's meaning.**
  - Observation 2: **words with similar row {they appear in similar context} they may have similar meanings.**
- Problem to solve:



- How can we fill this matrix?
  - Is Count a good idea?
  - **Remember the limitations of TF – IDF, so no.**
- What could be the solutions?

How can we fill this matrix?
Idea:
Instead of counting the appearance can we estimate the probability of the next word.

Towards Language Model

N-Gram Model

- {Approach1: $P_{\Omega} = \frac{\#\text{ Favourable Events}}{\#\text{ Total Events in }\Omega}$}
- **When N = 1 i.e. Unigram Model.**
  - For the unigram language model, we need to estimate
  - P(word) for every word in the text
    - $\mathbf{P(word)} = \frac{n_i}{N}$
    - $\mathbf{n_i}$ is the number of occurrences of word in the collection
    - **N** is the total number of word occurrences (i.e., tokens) in the collection

- **{Approach2: $P(A|B) = \frac{P(A \cap B)}{P(B)}$ (Conditional)}**
- **When N = 2 i.e. Bigram Model**
  - For the bigram language model, we additionally need to estimate
    - P(word | previous word) **for every pair of words** that **appear one after another**
      - $\mathbf{P(w|w_{i-1}) = \frac{n(w_{i-1},w_i)}{n(w_{i-1})}}$ **{Approach1}**
      - $\mathbf{n(w_{i-1}, w_i)}$ the number of occurrences of bigram $\mathbf{w_{i-1}, w_i}$ in the collection
      - $\mathbf{n(w_{i-1})}$ the number of occurrences of term $\mathbf{w_{i-1}}$ in the collection

Estimating Probabilities in N-Gram LM.

- Examples:
  - We are given a toy collection consisting of three documents
    - **d1 : "Frodo and Sam stabbed orcs"**
    - **d2 : "Sam chased the orc with the sword"**
    - **d3 : "Sam took the sword"**
  - **Estimating word probabilities for the unigram model:**

t_i	Frodo	Sam	orc	chased	sword	...
$P(t_i)$	1/16	3/16	2/16	1/16	2/16	...

  - Estimates : → **How many times** the word appears in the collection of documents?
    - Does not embed the semantics in a vector representations as context are not included in the estimation of probability.

Estimating Probabilities in N-Gram LM.

- Examples:
  - We are given a toy collection consisting of three documents
    - **d1 : "Frodo and Sam stabbed orcs"**
    - **d2 : "Sam chased the orc with the sword"**
    - **d3 : "Sam took the sword"**
- **Estimating the conditional probabilities for the bigram model:**

t_{i-1}, t_i	Frodo, chased	the, sword	the, orc	...
$P(t_i \mid t_{i-1})$	0	2/3	1/3	...

  - This is for N = 2, Estimates:→ How many times word have appeared in pair?
    - This approach tries to embed the context by considering neighboring words, this might get better result if we could consider a greater number of neighboring words i.e. increase N's values.
    - What will be the challenge if we increase N?

Challenges of N-gram LM.

- **Challenges:**
  - Language models have a major issue
    - The longer the phrase, the harder it is to estimate its true probability in language
      - E.g., P(“bilbo” | “frodo ran around house found ring”) = ?
    - Long phrases have very few appearances even in very large corpora
      - Impossible to compute reliable estimates of their conditional probabilities
      - This is why language models for N ≥ 3 are almost never used
      - They are **sparse**.
- Is there better way to estimate rather than traditional approach?
  - What about using neural network, they also estimate a probability(likelihood) in center?
    - **Towards Neural Embedding.**

Neural Embedding: Inutition.

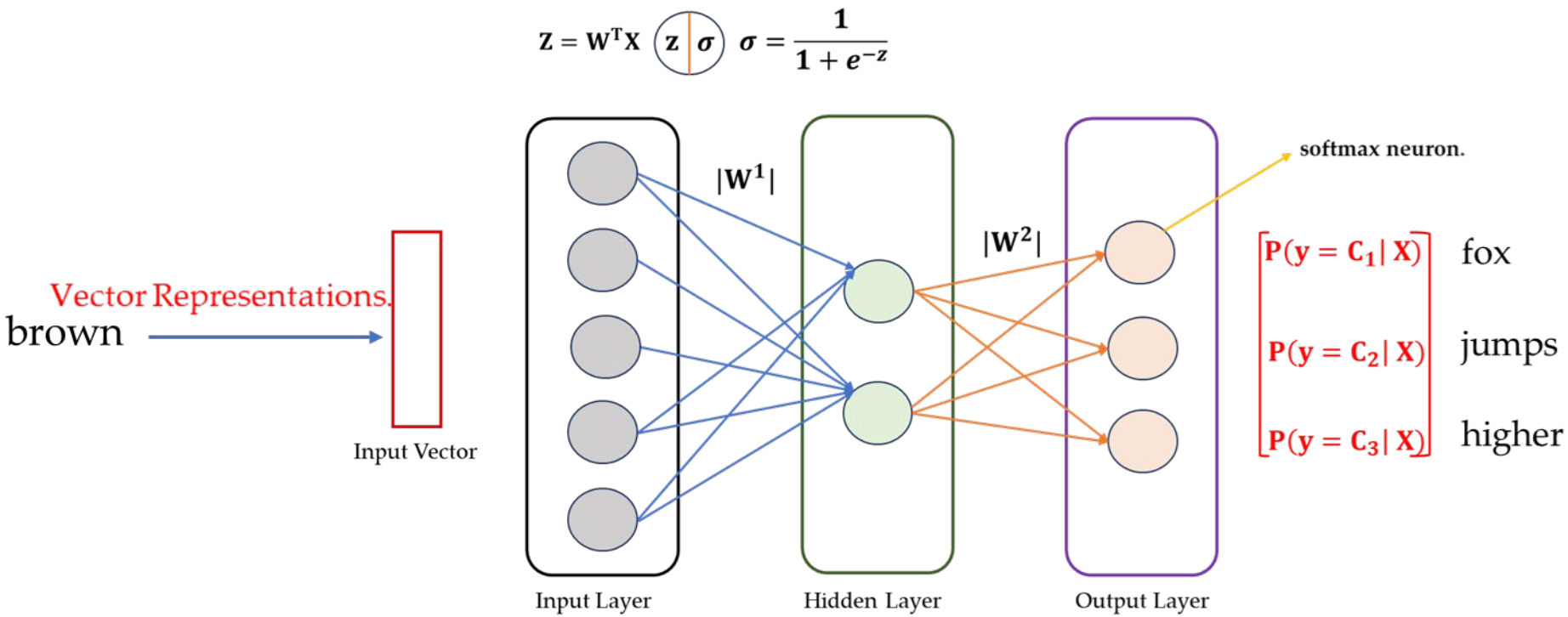


Fig: A Fully Connected Neural Network.

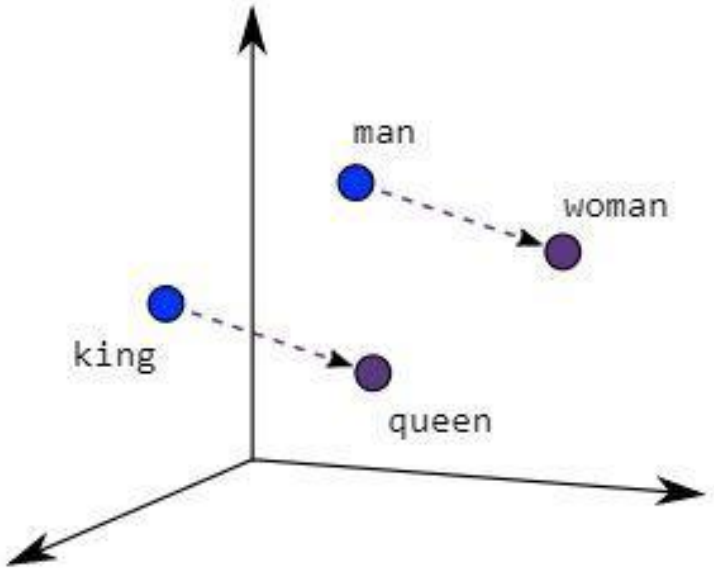
Can we use this to predict the probability of next word ?

Neural Word Embedding

“word2vec”

Word2Vec (Developed by Google (Mikolov et al., 2013)

- Two layer neural network to generate word embedding given a text corpus
- Preserves the relationship between words and deals with addition of new words in vocabulary
- Word2vec representation causes the words with similar meaning have similar embedding



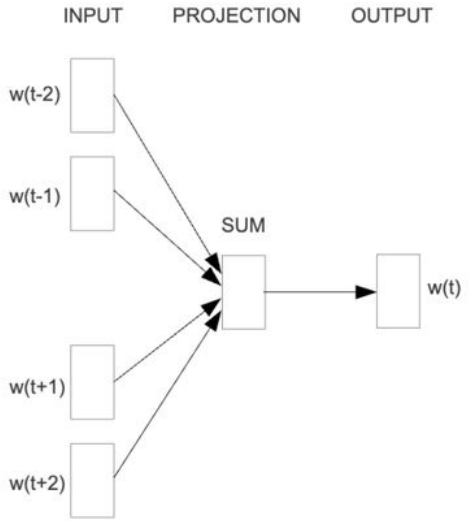
King - Man + Woman = Queen

Two variants of Word2Vec

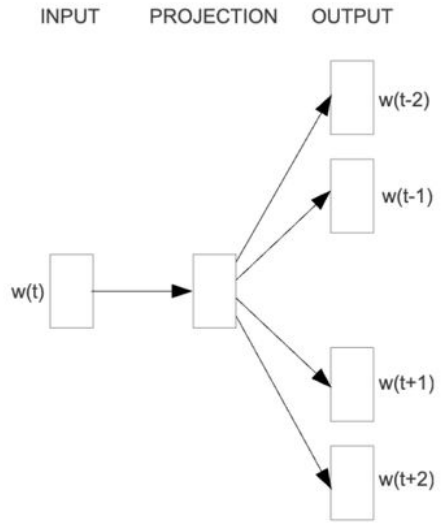
- Continuous Bag of Words (CBOW)
  - Predicts target word from context words
- Skip-gram
  - Predicts context words from target word

Both CBoW and Skip-gram use:

- Shallow neural network (2 layers)
- Trained on large text corpus
- Learn embeddings as a "side effect"



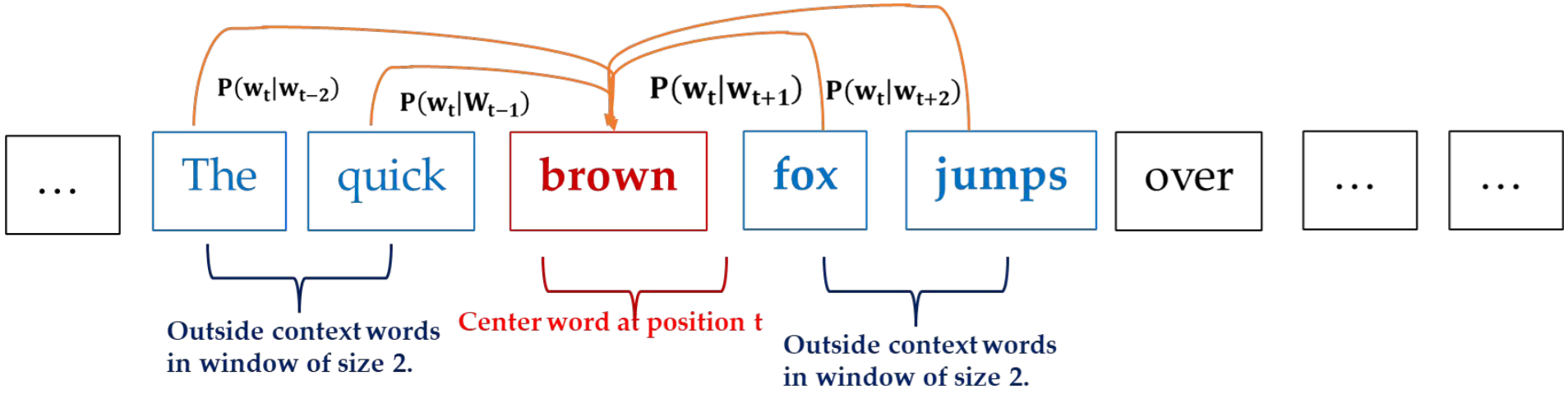
CBOW



Skip-gram

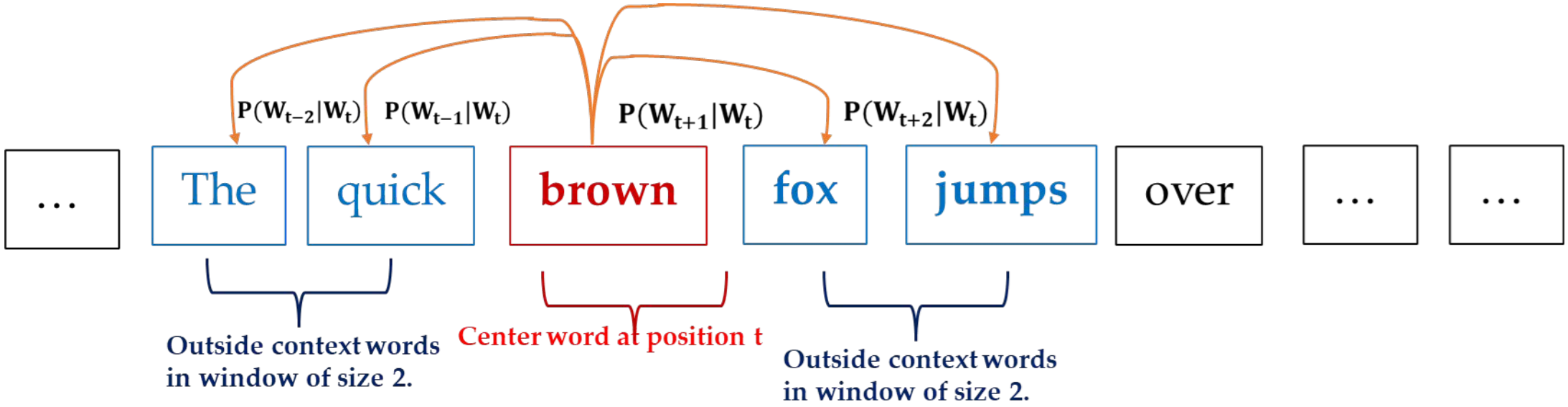
“word2vec” – variant: CBOW.

- CBOW – Continuous Bag of Words:
  - It is a neural network – based model in word2vec family.
  - Given a sequence of words, CBOW uses the context words within a window of size 2k (i.e. k words before and k words after the target) to predict the center word.



“word2vec” – variant: skip - gram.

- The idea: we want to use center words to predict their context words.
- Context: a fixed window of size 2k.



"CBOW": Dataset Generation.

- **Dataset Generation for "CBOW":**
  - Example Dataset: "the quick brown fox jumps over the lazy dog"
  - Step 1: we create a vocabulary of the unique words using tokenization:
    - **tokens** = ["the", "quick", "brown","fox", "jumps","over", "the", "lazy", "dog"]
    - Sort for consistency:
      - **sorted_tokens** = ["brown","dog","fox","jumps", "lazy", "over","quick","the"]
      - Assign index to each word:
  - Step 2: Create a one hot encoded vector based on tokens.

word	Index - Position	One – Hot Vector
brown	0	[1, 0, 0, 0, 0, 0, 0, 0]
dog	1	[0, 1, 0, 0, 0, 0, 0, 0]
fox	2	[… …]
jumps	3	[… …]
lazy	4	[… …]
over	5	[0, 0, 0, 0, 0, 1, 0, 0]
quick	6	[… …]
the	7	[0, 0, 0, 0, 0, 0, 0, 1]

"CBOW": Dataset Generation.

- **Dataset Generation for "CBOW":**
  - Step 3: set context window size.
    - **window_size, k = 2**
  - Step 4: Generate CBOW Training samples.

Context Words	Index or Position Representations {Context}	Target	Index or Position Representations {Target}
[the, quick, fox, jumps]	[7, 6, 2, 3]	brown	[0]
[quick, brown, jumps, over]	[6, 0, 3, 5]	fox	[2]
[brown, fox, over, the]	[0, 2, 5, 7]	jumps	[3]
[fox, jumps, the, lazy]	[2, 3, 7, 4]	over	[5]
[jumps, over, lazy, dog]	[3, 5, 4, 1]	the	[7]

  - Based on the index created as per sorted vocabulary the whole sentence can be:
  - **"the quick brown fox jumps over the lazy dog" → [7, 6, 0, 2, 3, 5, 7, 4, 1]**

"CBOW": Look Up Table.

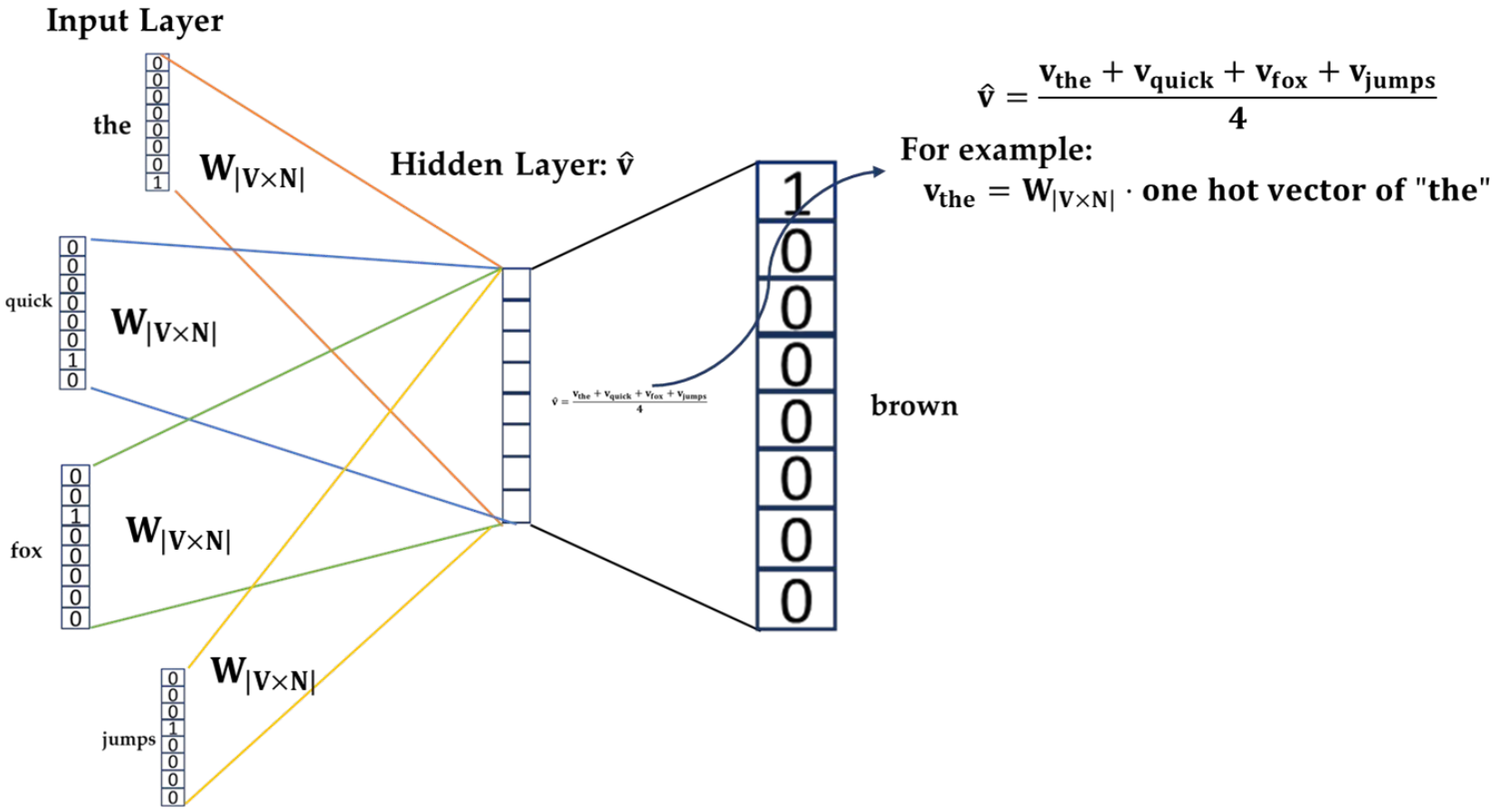
word	Index - Position	One – Hot Vector
brown	0	[1, 0, 0, 0, 0, 0, 0, 0]
dog	1	[0, 1, 0, 0, 0, 0, 0, 0]
fox	2	[... ...]
jumps	3	[... ...]
lazy	4	[... ...]
over	5	[0, 0, 0, 0, 0, 1, 0, 0]
quick	6	[... ...]
the	7	[0, 0, 0, 0, 0, 0, 0, 1]

Context Words	Index or Position Representations {Context}	Target	Index or Position Representations {Target}
[the, quick, fox, jumps]	[7, 6, 2, 3]	brown	[0]
[quick, brown, jumps, over]	[6, 0, 3, 5]	fox	[2]
[brown, fox, over, the]	[0, 2, 5, 7]	jumps	[3]
[fox, jumps, the, lazy]	[2, 3, 7, 4]	over	[5]
[jumps, over, lazy, dog]	[3, 5, 4, 1]	the	[7]

Fig: Similar Look Up Table is Created for whole Vocabulary.

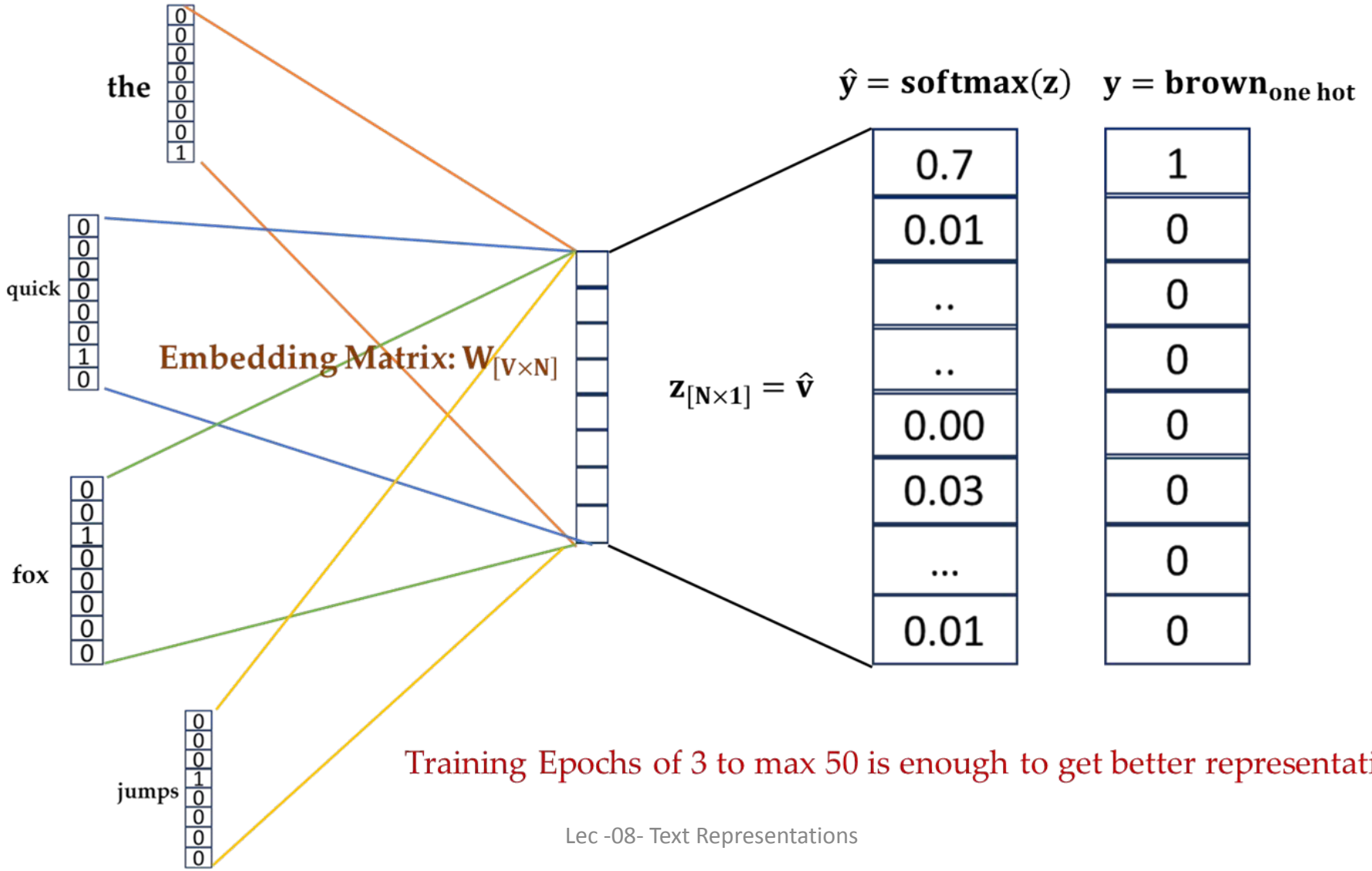
Training “CBOW”.

- Example Input:
  - the quick brown fox …….
  - Window size k = 2
- Position – 2
  - Target = “brown”
  - Context = [the, quick, fox, jumps]



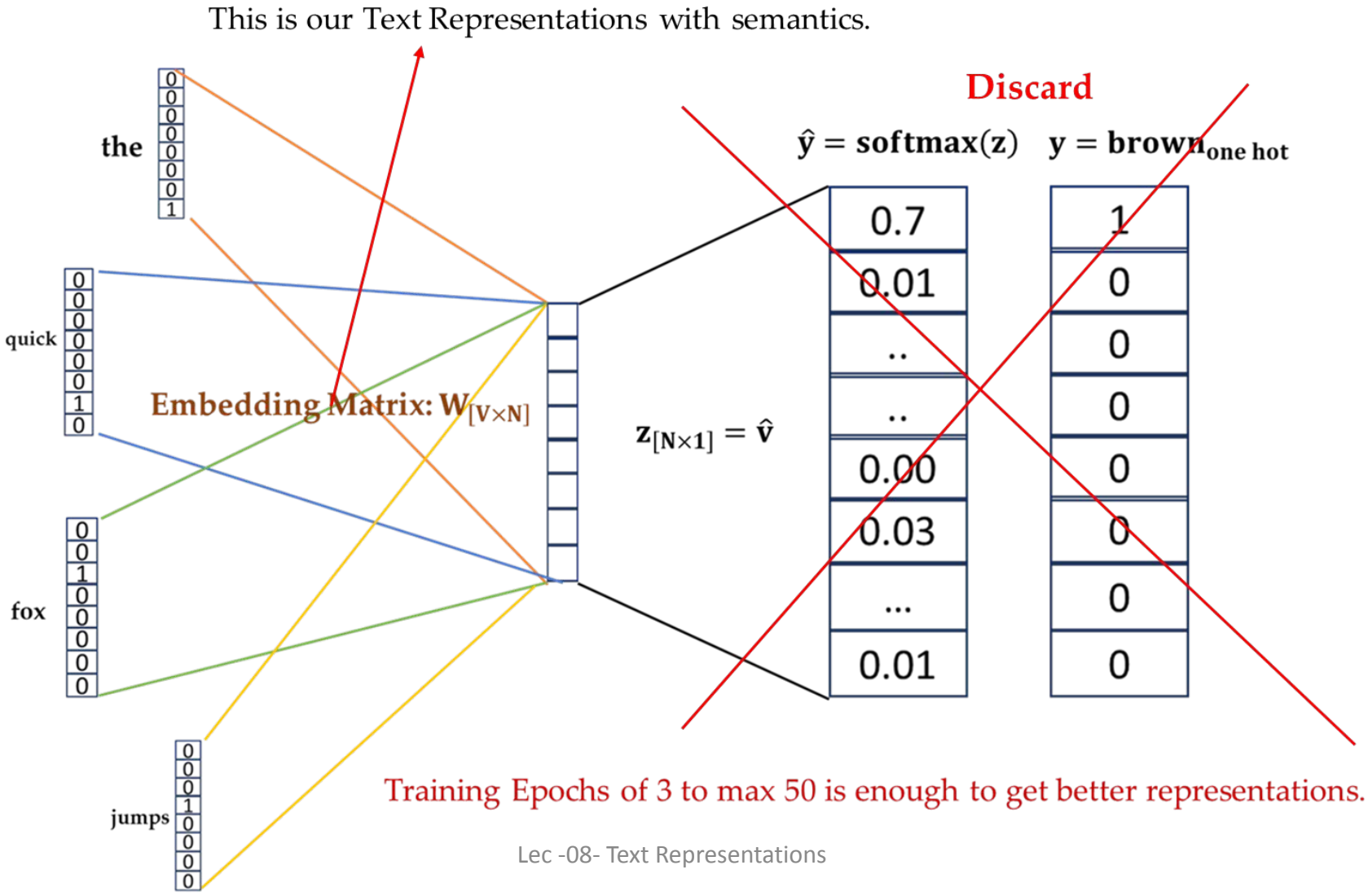
- $W_{|V\times N|}$ → **Embedding Matrix**
  - Here:
    - **V → len of vocabulary**
    - **N → number of neuron in Hidden Layer kept less than V**
    - **Initialized Randomly.**

Train using "Gradient Descent".



Training Epochs of 3 to max 50 is enough to get better representations.

After Training.



Why "word2vec"?

- Word2Vec embeddings capture **semantic relationships** between words — so well,
  - in fact, that you can do **word analogy arithmetic** like this famous one:
  - king − man + woman ≈ queen i.e. for the question:
    - man is to king as woman is to ...?
      - Probability for queen is the highest in the corpus of English language …

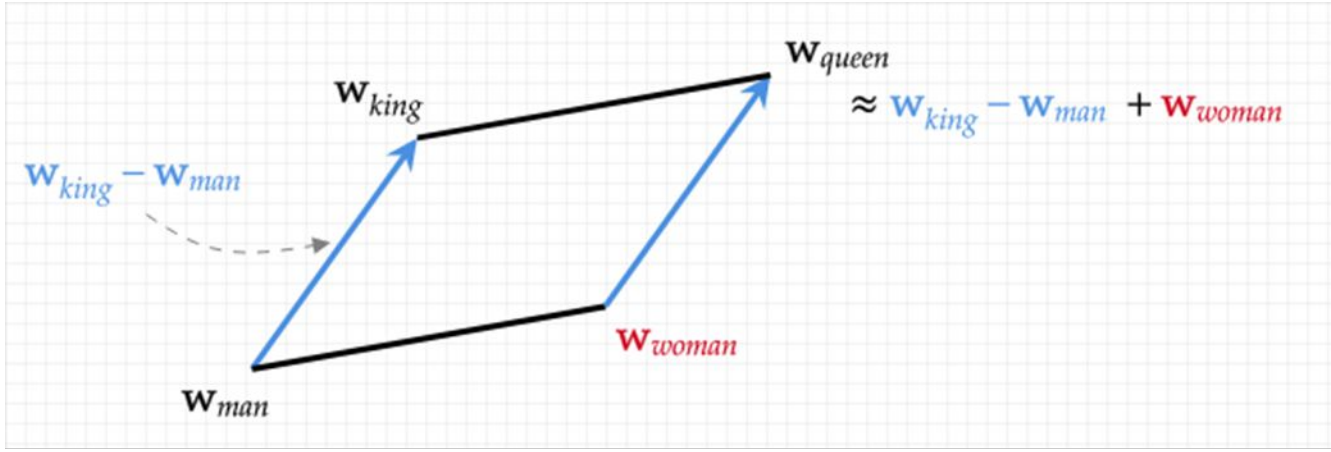


Fig: word2vec Word Embedding Vector Space.

“word2vec” in Practice.

- In practice, **we usually don't train Word2Vec from scratch,**
  - Instead, we use **pretrained Word2Vec models**, especially the popular one trained by Google.
- **Pre – trained word2vec by Google:**
  - Corpus: Trained on Google News dataset (approx. 10 billion words)
    - Vocab size – approx. 3 million word/phrases.
  - Dimension available: 50D, 100D, 200D, 300D
    - **300D is a most popular and is best suit for capturing word relationships.**
- **Using Genism to Load Pre – trained word2vec:**

```
# Load pretrained Google News Word2Vec model (300 dimensional)
# Download from: https://code.google.com/archive/p/word2vec/
# File: GoogleNews-vectors-negative300.bin.gz (about 1.5GB)
from gensim.models import KeyedVectors

model_path = 'GoogleNews-vectors-negative300.bin.gz'
print("Model loaded!")

# Check vector for a word
vector = word2vec['computer']
print("Vector for 'computer':", vector[:10])  # print first 10 dims

# Check similarity
print("Similarity between 'king' and 'queen':", word2vec.similarity('king', 'queen'))

# Word analogy: king - man + woman ≈ queen
result = word2vec.most_similar(positive=['woman', 'king'], negative=['man'], topn=1)
print("Result of analogy (king - man + woman):", result)
```

Alternate to “word2vec”.

Model	Description	Dimensions	Trained On
word2vec	**Predictive Model (skip –gram / CBOW)**	**100, 300**	**Google News**
GloVe	Count – based – matrix factorizations	50, 100, 200, 300	Common Crawl, Wikipedia
FastText	Extends word2vec using sub word info	300	Common Crawl, Wikipedia
ELMo, BERT	Contextual word embeddings {Transformer based → LLM are based on this.}	Variable (million +)	Huge Corpora

Thank You